

0/1 Knapsack Problem

Last Updated : 23 Jul, 2025



Given n items where each item has some weight and profit associated with it and also given a bag with capacity W , [i.e., the bag can hold at most W weight in it]. The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.

Note: The constraint here is we can either put an item completely into the bag or cannot put it at all [It is not possible to put a part of an item into the bag].

Input: $W = 4$, $profit[] = [1, 2, 3]$, $weight[] = [4, 5, 1]$

Output: 3

Explanation: There are two items which have weight less than or equal to 4. If we select the item with weight 4, the possible profit is 1. And if we select the item with weight 1, the possible profit is 3. So the maximum possible profit is 3. Note that we cannot put both the items with weight 4 and 1 together as the capacity of the bag is 4.

Input: $W = 3$, $profit[] = [1, 2, 3]$, $weight[] = [4, 5, 6]$

Output: 0

Table of Content

- [\[Naive Approach\] Using Recursion \$O\(2^n\)\$ Time and \$O\(n\)\$ Space](#)
- [\[Better Approach 1\] Using Top-Down DP \(Memoization\) - \$O\(n \times W\)\$ Time and Space](#)
- [\[Better Approach 2\] Using Bottom-Up DP \(Tabulation\) - \$O\(n \times W\)\$ Time and Space](#)
- [\[Expected Approach\] Using Bottom-Up DP \(Space-Optimized\) - \$O\(n \times W\)\$ Time and \$O\(W\)\$ Space](#)
- [Problems based on 0-1 Knapsack](#)

[Naive Approach] Using Recursion $O(2^n)$ Time and $O(n)$ Space

A simple solution is to consider all subsets of items and calculate the total weight and value of all subsets. Consider the only subsets whose total

weight is smaller than W . From all such subsets, pick the subset with maximum value.

Optimal Substructure: To consider all subsets of items, there can be two cases for every item.

- **Case 1:** The item is included in the optimal subset.
- **Case 2:** The item is not included in the optimal set.

Follow the below steps to solve the problem:

The maximum value obtained from 'n' items is the max of the following two values.

- Case 1 (pick the n^{th} item): Value of the n^{th} item + maximum value obtained by remaining (n-1) items and remaining weight i.e. (W -weight of the n^{th} item).
- Case 2 (don't pick the n^{th} item): Maximum value obtained by (n-1) items and W weight.
- If the weight of the ' n^{th} ' item is greater than ' W ', then the n^{th} item cannot be included and **Case 2** is the only possibility.

C++

Java

Python

C#

JavaScript

```
#include <bits/stdc++.h>
using namespace std;

// Returns the maximum value that
// can be put in a knapsack of capacity W
int knapsackRec(int W, vector<int> &val, vector<int> &wt, int n) {

    // Base Case
    if (n == 0 || W == 0)
        return 0;

    int pick = 0;

    // Pick nth item if it does not exceed the capacity of
    knapsack
    if (wt[n - 1] <= W)
        pick = val[n - 1] + knapsackRec(W - wt[n - 1], val, wt, n
```

```

- 1);

    // Don't pick the nth item
    int notPick = knapsackRec(W, val, wt, n - 1);

    return max(pick, notPick);
}

int knapsack(int W, vector<int> &val, vector<int> &wt) {
    int n = val.size();
    return knapsackRec(W, val, wt, n);
}

int main() {
    vector<int> val = {1, 2, 3};
    vector<int> wt = {4, 5, 1};
    int W = 4;

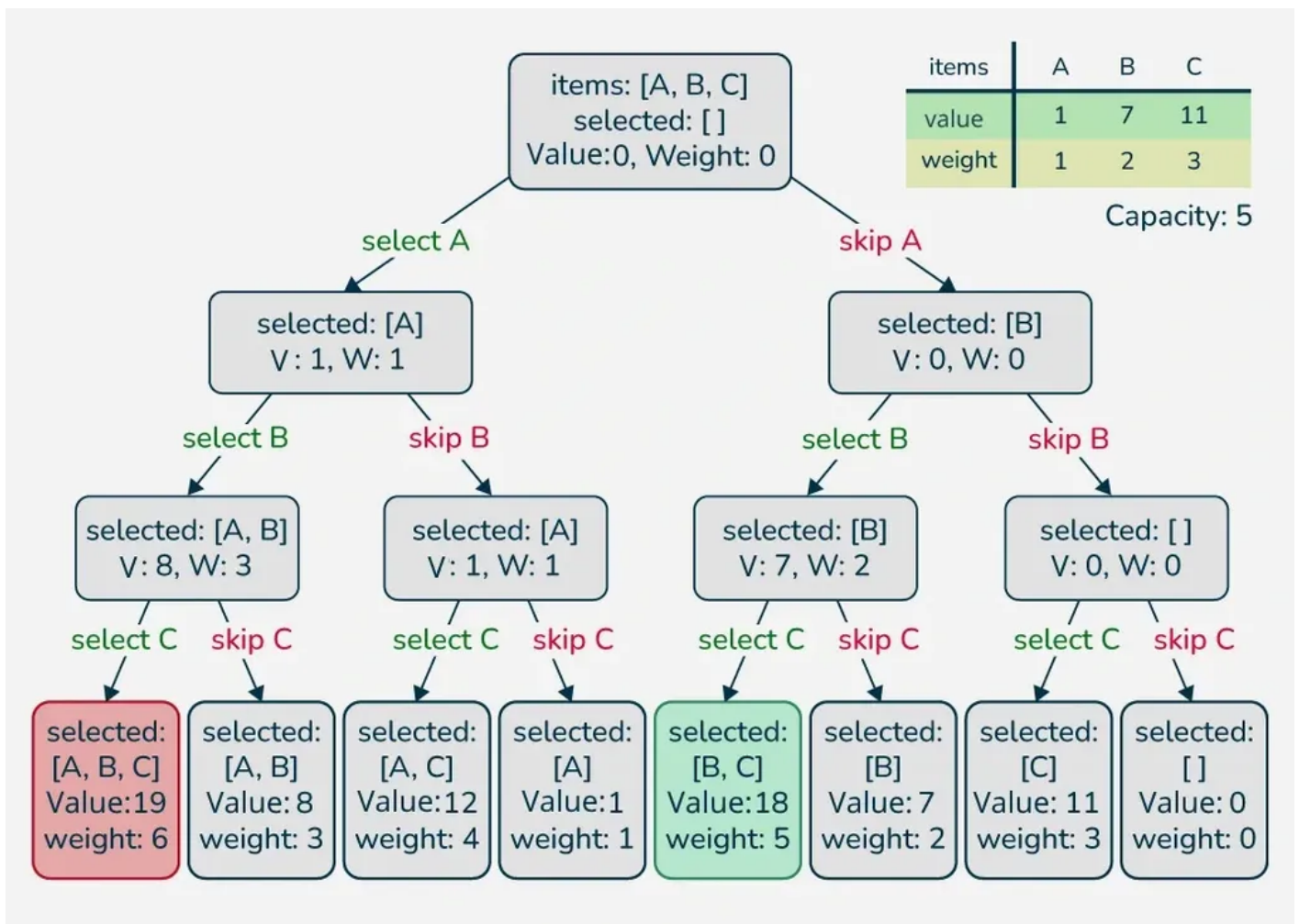
    cout << knapsack(W, val, wt) << endl;
    return 0;
}

```

Output

3

Below is an example run of the above implementation.



[Better Approach 1] Using Top-Down DP (Memoization)- $O(n \times W)$ Time and Space

Note: The above function using recursion computes the same subproblems again and again. See the following recursion tree, $K(1, 1)$ is being evaluated twice.

Recursion tree for 0/1 Knapsack

weight[] = { 1, 1, 1 } W = 2 value[] = { 10, 20, 30 }

Remaining Items

Capacity

K(3, 2)

✓ Choose
✗ Not Choose

K(2, 1)

K(2, 2)

K(1, 0)

K(1, 1)

K(1, 1)

K(1, 2)

K(0, 0)

K(0, 1)

K(0, 0)

K(0, 1)

K(0, 1)

K(0, 2)

As there are repetitions of the same subproblem again and again we can implement the following idea to solve the problem.

If we get a subproblem the first time, we can solve this problem by creating a 2-D array that can store a particular state (n, w). Now if we come across the same state (n, w) again instead of calculating it again we can directly return its result stored in the table in constant time.

C++ Java Python C# JavaScript

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Returns the maximum value that
```

```
// can be put in a knapsack of capacity W
```

```
int knapsackRec(int W, vector<int> &val, vector<int> &wt, int n,
```

```

vector<vector<int>> &memo)
{

    // Base Case
    if (n == 0 || W == 0)
        return 0;

    // Check if we have previously calculated the same subproblem
    if(memo[n][W] != -1)
        return memo[n][W];

    int pick = 0;

    // Pick nth item if it does not exceed the capacity of
    knapsack
    if (wt[n - 1] <= W)
        pick = val[n - 1] + knapsackRec(W - wt[n - 1], val, wt, n
- 1, memo);

    // Don't pick the nth item
    int notPick = knapsackRec(W, val, wt, n - 1, memo);

    // Store the result in memo[n][W] and return it
    return memo[n][W] = max(pick, notPick);
}

int knapsack(int W, vector<int> &val, vector<int> &wt) {
    int n = val.size();

    // Memoization table to store the results
    vector<vector<int>> memo(n + 1, vector<int>(W + 1, -1));

    return knapsackRec(W, val, wt, n, memo);
}

int main() {
    vector<int> val = {1, 2, 3};
    vector<int> wt = {4, 5, 1};
    int W = 4;

    cout << knapsack(W, val, wt) << endl;
    return 0;
}

```

```
}
```

Output

```
3
```

[Better Approach 2] Using Bottom-Up DP (Tabulation) - $O(n \times W)$ Time and Space

There are two parameters that change in the recursive solution and these parameters go from 0 to n and 0 to W . So we create a 2D `dp[][]` array of size $(n+1) \times (W+1)$, such that `dp[i][j]` stores the maximum value we can get using i items such that the knapsack capacity is j .

- We first fill the known entries when m is 0 or n is 0.
- Then we fill the remaining entries using the recursive formula.

For each item i and knapsack capacity j , we decide whether to pick the item or not.

- **If we don't pick the item:** `dp[i][j]` remains same as the previous item, that is `dp[i - 1][j]`.
- **If we pick the item:** `dp[i][j]` is updated to `val[i] + dp[i - 1][j - wt[i]]`.

C++ Java Python C# JavaScript

```
#include <bits/stdc++.h>
using namespace std;

// Returns the maximum value that
// can be put in a knapsack of capacity W
int knapsack(int W, vector<int> &val, vector<int> &wt) {
    int n = wt.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1));

    // Build table dp[][] in bottom-up manner
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= W; j++) {

            // If there is no item or the knapsack's capacity is 0
```

```

        if (i == 0 || j == 0)
            dp[i][j] = 0;
        else {
            int pick = 0;

            // Pick ith item if it does not exceed the capacity
of knapsack
            if(wt[i - 1] <= j)
                pick = val[i - 1] + dp[i - 1][j - wt[i - 1]];

            // Don't pick the ith item
            int notPick = dp[i - 1][j];

            dp[i][j] = max(pick, notPick);
        }
    }
}
return dp[n][W];
}

int main() {
    vector<int> val = {1, 2, 3};
    vector<int> wt = {4, 5, 1};
    int W = 4;

    cout << knapsack(W, val, wt) << endl;
    return 0;
}

```

Output

3

[Expected Approach] Using Bottom-Up DP (Space-Optimized) - $O(n \times W)$ Time and $O(W)$ Space

For calculating the current row of the $dp[]$ array we require only previous row, but if we start traversing the rows from right to left then it can be done with a single row only


```
#include <bits/stdc++.h>
using namespace std;

// Function to find the maximum profit
int knapsack(int W, vector<int> &val, vector<int> &wt) {

    // Initializing dp vector
    vector<int> dp(W + 1, 0);

    // Taking first i elements
    for (int i = 1; i <= wt.size(); i++) {

        // Starting from back, so that we also have data of
        // previous computation of i-1 items
        for (int j = W; j >= wt[i - 1]; j--) {
            dp[j] = max(dp[j], dp[j - wt[i - 1]] + val[i - 1]);
        }
    }
    return dp[W];
}

int main() {
    vector<int> val = {1, 2, 3};
    vector<int> wt = {4, 5, 1};
    int W = 4;

    cout << knapsack(W, val, wt) << endl;
    return 0;
}
```

Output

3

Problems based on 0-1 Knapsack

- [Unbounded Knapsack](#)
- [Double Knapsack](#)

- [Subset Sum](#)
- [Coin Change - Count Ways](#)
- [Coin Change - Min Coins](#)
- [Partition Equal Subset Sum](#)
- [Minimum Subset Sum Difference](#)
- [Maximum Segments](#)

Knapsack Problem - Approach to write the code (Dynamic Programming)

 Comment

More info 

Advertise with us

Similar Reads

Basics & Prerequisites



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information